Antigravity Work Plan: Hostel Room Allocation Fairness Auditor

Team: Zaid Attar, Arpit Tiwari, Trushant Chilvery, Aryan Bhure

1. What to build

Build a web app that audits a hostel room allocation. The user supplies three CSV files: students, rooms, and the actual allocation (who really got which room). The system computes what a fair allocation would have been using a weighted policy, compares it with the actual allocation, flags violations, computes a fairness score, and shows everything in a React dashboard. A MySQL database stores the data and supports basic management: creating tables, adding, editing and deleting students and rooms, and filtering them (for example, only OBC students).

The data visualization dashboard is the most important part of the project. The database part is deliberately minimal. This is an auditor, not an allocator: the fair allocation exists only as the reference to compare the actual allocation against.

Feature priorities (read before building)

The dashboard comes first. If time runs short, cut from the bottom of this list.

- Must have: the explainability panel (click a student to see their priority score split into factors, their expected room, their actual room and the gap) and the per-group flagged percent on the bias chart (a toggle between average deviation and percent flagged).
- Cheap, add if any time is left: the corrected allocation CSV export and the Student Lookup search box.
- Cut for now: the CSV upload and validation screen. The three CSVs are placed in the data/ folder, and load_csv.py runs the basic data checks and prints warnings in the terminal.

2. Rules for the agent

- Work one phase at a time (Section 9). After each phase, run its checks, summarize what works, and wait for approval before starting the next phase.
- Keep the code simple and readable. This is a college mini project, so prefer plain functions and clear comments over clever abstractions.

- Do not change the stack in Section 3 or add features outside this plan: no login, no machine learning, no deployment work, no warden, fee or complaint modules.
- Put all audit logic in one backend module (audit.py) made of pure functions, so it can be tested without a database or a server.
- Use a fixed random seed in the data generator so results are reproducible.
- If something in this plan is ambiguous, write the assumption in the README and continue. Ask only if the choice would change the stack or the data model.

3. Tech stack

- Backend: Python 3.11+, FastAPI, SQLAlchemy with PyMySQL, pandas, numpy, scipy, pytest
- Database: MySQL 8
- Frontend: React with Vite, Recharts for all charts, plain CSS (no UI framework)
- Config: a .env file holding the database URL, with a .env.example committed

4. Project structure

```
hostel-auditor/
  backend/
    app/
      main.py          # FastAPI app and routes
      db.py            # SQLAlchemy engine and session
      crud.py          # student, room, allocation queries
      audit.py         # pure audit functions (no DB access)
      validation.py    # basic data checks, called by load_csv.py
    scripts/
      schema.sql
      generate_data.py
      load_csv.py
    tests/
      test_audit.py
    requirements.txt
  data/
    students.csv
    rooms.csv
    allocation.csv
  frontend/
    src/
```

```
      App.jsx
      api.js
      components/
  README.md
```

5. Data model

5.1 Input CSV files

- students.csv: student_id, name, gender (M or F), year (1 to 4), cgpa (0 to 10), category (General, OBC, SC, ST, EWS), distance_km, special_needs (0 or 1)
- rooms.csv: room_id, hostel, gender (M or F), floor (0 is ground), mess_distance_m, accessible (0 or 1)
- allocation.csv: student_id, room_id. One row per allocated student. A student missing from this file is unallocated.

Assumptions: each row in rooms.csv is one allocatable bed. Boys and girls are audited in separate pools by gender. Rank 1 always means best.

5.2 MySQL tables

```
CREATE TABLE students (
  student_id VARCHAR(20) PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  gender CHAR(1) NOT NULL,
  year TINYINT NOT NULL,
  cgpa DECIMAL(4,2) NOT NULL,
  category VARCHAR(20) NOT NULL,
  distance_km INT NOT NULL,
  special_needs TINYINT(1) NOT NULL DEFAULT 0
);

CREATE TABLE rooms (
  room_id VARCHAR(20) PRIMARY KEY,
  hostel VARCHAR(50) NOT NULL,
  gender CHAR(1) NOT NULL,
  floor TINYINT NOT NULL,
  mess_distance_m INT NOT NULL,
  accessible TINYINT(1) NOT NULL DEFAULT 0
);

CREATE TABLE allocations (
  student_id VARCHAR(20) PRIMARY KEY,
  room_id VARCHAR(20) NOT NULL UNIQUE,
```

```
  FOREIGN KEY (student_id) REFERENCES students(student_id) ON DELETE CASCADE,
  FOREIGN KEY (room_id) REFERENCES rooms(room_id) ON DELETE CASCADE
);

CREATE TABLE policy_rules (
  rule_key VARCHAR(40) PRIMARY KEY,
  value DECIMAL(6,3) NOT NULL
);

CREATE TABLE audit_results (
  student_id VARCHAR(20) PRIMARY KEY,
  priority_score DECIMAL(6,2),
  expected_room_id VARCHAR(20),
  actual_room_id VARCHAR(20),
  deviation INT,
  status VARCHAR(30),
  hard_rule_violation TINYINT(1),
  audited_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Default policy_rules values: w_cgpa 0.35, w_distance 0.20, w_year 0.20, w_category 0.10, w_special_needs 0.15, w_room_mess 0.60, w_room_floor 0.40, tolerance_pct 10.

Category bonus values live in a constants dict inside audit.py with a comment: General 0, OBC 0.5, EWS 0.5, SC 1.0, ST 1.0.

6. Audit algorithm (audit.py)

1. Priority score. Normalize each factor to 0 to 1: cgpa divided by 10, distance_km divided by the maximum distance in the data, year divided by 4, the category bonus value, and special_needs as 0 or 1. Priority score = 100 times the weighted sum divided by the sum of the weights.
2. Room quality. Quality = (w_room_mess times (1 minus mess_distance divided by max mess distance) plus w_room_floor times (1 minus floor divided by max floor)) divided by the sum of the two weights, scaled to 0 to 100. Guard against dividing by zero when a maximum is 0.
3. Pools. Run everything below separately for the M pool and the F pool. Rank rooms by quality descending (tie-break by room_id). Rank students by priority descending (tie-break by higher cgpa, then student_id).
4. Expected allocation. First, special-needs students in priority order each take the best remaining accessible room. Then the remaining students in priority order take the best remaining rooms in order. If students outnumber rooms, the lowest-priority students get no expected room. If rooms outnumber students, the extra rooms stay empty.

5. Deviation. For each student, actual_room_rank is the rank of their actual room within the pool (none if unallocated) and expected_room_rank is the rank of their expected room. Deviation = actual_room_rank minus expected_room_rank. Negative means a better room than deserved. Positive means a worse room.
6. Status. Tolerance = tolerance_pct percent of the pool's room count, at least 1. FAIR if the absolute deviation is within tolerance. OVER_FAVORED if deviation is below minus tolerance. UNDER_SERVED if deviation is above tolerance. UNALLOCATED if the student has no actual room but has an expected one.
7. Hard rule. A special-needs student in a non-accessible actual room gets hard_rule_violation = true (violation type SPECIAL_NEEDS_NOT_ACCESSIBLE) regardless of deviation. A student is flagged if the status is not FAIR or the hard rule is violated.
8. Plot values. Compute priority_percentile and room_quality_percentile (of the actual room), both 0 to 100 with 100 as best, for plotting.
9. Fairness score. Compute the Spearman correlation (scipy.stats.spearmanr) between priority score and actual room quality across allocated students, per pool and overall. fairness_score = max(0, rho) times 100, rounded to an integer. Also return compliance_rate = FAIR students divided by total students.
10. Displacement pairs for the who-got-it-versus-who-should-have table. For each flagged student, find the student who was expected to hold their actual room. Return room_id, should_have (that student), got_it (the actual holder), and both priority scores. Sort by priority gap descending.
11. Group stats. For category, year, distance band (0 to 100, 101 to 300, 301 to 600, over 600 km) and gender, return count, average deviation, flagged count and flagged percent.
12. Explainability. For one student, return each factor's weighted contribution to the priority score, the expected room, the actual room, the deviation, the status, and a plain-English reason string.

The corrected allocation is simply the expected allocation from step 4, exported as a CSV.

7. API endpoints

- GET /health
- No upload endpoint: data is loaded from the data/ folder by backend/scripts/load_csv.py
- GET /students with filters: category, year, gender, min_distance, max_distance, special_needs, search, page
- POST /students, PUT /students/{id}, DELETE /students/{id}
- GET /rooms with filters, plus POST, PUT and DELETE

- GET /allocations, PUT /allocations/{student_id}
- GET /policy, PUT /policy
- GET /audit: runs the audit live. Optional query parameters override the policy weights without saving them, so the dashboard sliders can call it. With save=true it also writes audit_results. Returns summary KPIs, per-student rows, group stats and displacement pairs.
- GET /audit/student/{id}: the explainability breakdown
- GET /export/corrected-allocation.csv

Validation rules, run inside load_csv.py and printed in the terminal (no UI). Errors that stop loading: duplicate IDs, cgpa outside 0 to 10, unknown gender or category, an allocation referencing an unknown student or room, a room assigned to more than one student, a student placed in a room of the other gender. Warnings that still load: students with no allocation, and rooms with no student.

8. Frontend

Use tabs: Dashboard, Students, Rooms, and Student Lookup (optional).

Dashboard layout: policy weight sliders in a left sidebar, content on the right. Sliders are debounced by about 300 ms and call GET /audit with the weights as overrides.

- KpiCards: fairness score, violations flagged, allocations audited, compliance rate
- PriorityScatter: Recharts ScatterChart. X is priority percentile, Y is room quality percentile, with a dashed diagonal ReferenceLine as the ideal. Three series: fair (blue circles), over-favored (red triangles), under-served (amber squares). Use distinct shapes so the meaning does not rely on color alone. Hovering a point shows the student ID and scores, and clicking opens the explainability panel.
- BiasBars: horizontal BarChart of average deviation, with a selector for category, year, distance band or gender, and a toggle between average deviation and percent flagged. Highlight bars that exceed the tolerance in red and use gray for the rest.
- ViolationsTable: the who-got-it-versus-who-should-have pairs, sortable.
- StudentDetail drawer: the explainability breakdown for the clicked student.

Students and Rooms tabs: a table with filter controls, an add form, inline edit and delete. Changing data should refresh the audit.

Student Lookup tab (optional, cheap): enter a student ID and see that student's priority score, expected room, actual room, status and reason.

9. Phases and workflow

Phase 0: Setup

Scaffold the folders from Section 4, requirements.txt, the Vite app, .env.example and a README skeleton. Check: GET /health returns ok, and the frontend dev server starts.

Phase 1: Data and database

Write schema.sql, generate_data.py and load_csv.py. The generator must be seeded and create 300 students (150 M and 150 F), 300 rooms (150 per gender) and an actual allocation.

- Students: cgpa roughly normal around 7.5, clamped to 5 to 10. Distance skewed between 5 and 1500 km. Years 1 to 4. Categories about 45 percent General, 27 percent OBC, 12 percent SC, 6 percent ST, 10 percent EWS. About 4 percent special needs.
- Rooms: floors 0 to 4, mess distance 20 to 400 m, accessible = 1 for ground-floor rooms.
- Allocation: start from the ideal allocation computed with the same algorithm, then add small neighbor swaps that stay within tolerance. Then plant violations: swap about 8 percent of students across large rank gaps within a pool, move 3 special-needs students into non-accessible rooms, and leave 2 students unallocated. Plant a mild systematic bias so one category (a BIASED_CATEGORY constant, default OBC) is more likely to be picked as the under-served student. This gives the bias chart something real to show.

load_csv.py must also run the basic data checks from Section 7 (duplicate IDs, a room assigned twice, unknown categories, a student placed in a room of the other gender) and print errors and warnings in the terminal before loading. Check: load_csv.py fills the tables, prints row counts and prints the data check results.

Phase 2: Audit engine and tests

Implement audit.py from Section 6 and write tests in test_audit.py using a tiny hand-made example of 6 students and 6 rooms. Required cases: a perfect allocation gives zero flagged students and rho of 1. Swapping the top and bottom students flags both. A special-needs student in a non-accessible room triggers the hard rule. A student missing from the allocation is UNALLOCATED. Check: pytest passes.

Phase 3: API for reading and auditing

Build db.py, crud.py and the routes for GET /audit, GET /audit/student/{id}, GET /policy, PUT /policy, the read endpoints with filters, and the corrected-allocation export. Check: the FastAPI docs page (/docs) works, and on the generated data the audit returns a

fairness score roughly between 80 and 95 with about 8 to 12 percent of students flagged.

Phase 4: Dashboard

Build the Dashboard tab from Section 8: sliders, KPI cards, scatter, bias bars, violations table and the explainability drawer. This is the priority phase, so spend the most care here. Check: moving a slider visibly changes the scatter and the fairness score, and clicking a red point opens its explanation.

Phase 5: Database pages

Build the Students and Rooms tabs with filtering, add, edit and delete, plus allocation edits. Check: filtering to OBC shows only OBC students, and adding or deleting a student changes the audit totals.

Phase 6: Extras and polish

Add the corrected allocation download button, the Student Lookup tab, and loading and error states (do not build a CSV upload screen). Finish the README with setup steps for MySQL, backend and frontend, how to regenerate the data, and the assumptions made. Check: a fresh clone can be set up by following the README alone.

10. Definition of done

- The three CSVs load into MySQL, and the dashboard works end to end on the generated data.
- The audit flags the planted violations, and the bias chart shows the planted bias.
- Changing policy weights updates the dashboard live.
- Students and rooms can be created, edited, deleted and filtered from the UI.
- Clicking any flagged student explains in plain English why they were flagged.
- The corrected allocation can be exported as a CSV.
- audit.py has passing tests, and the README explains how to run everything.
- The report states clearly that the demo data is synthetic and the policy weights are example values.